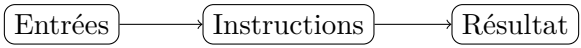
CHAPITRE 9

Python et algorithmes

Premiers pas au lycée. Lire, écrire et tester un petit programme, puis expliquer son résultat. Prérequis : calcul sur les entiers et notion de fonction.

Synthèse COMIMa : documentation Python 3.14, CodEx et références algorithmiques citées en fin de chapitre. Adaptations signalées.



9.1 Exemple introductif : calculer une aire

La documentation Python utilise ce calcul [P1] :

```
width = 20
height = 5 * 9
print(width * height)
```

L'affectation `=` associe une valeur à un nom. Ici, `height` vaut 45 et le programme affiche 900. Chaque ligne utilise les valeurs obtenues jusque-là : `=` n'est pas une équation à résoudre [P2].

9.2 Valeurs et opérations

Types utiles : `int` (entier), `float` (nombre à virgule), `str` (texte), `bool` (`True` ou `False`). Le séparateur décimal est un point ; un flottant représente généralement un réel de façon approchée [P1].

`+`, `-`, `*` et `/` servent aux opérations usuelles ; `//` donne le quotient entier inférieur, `%` le reste, et `**` une puissance. Pour des entiers $a \geq 0$ et $b > 0$, on retrouve $a = bq + r$ avec $0 \leq r < b$. Attention : `^` n'est pas la puissance en Python [P1].

9.3 Choisir et répéter

Définition 9.1. Un *test* produit un booléen. On compare avec `==`, `!=`, `<`, `<=`, `>` et `>=` ; `and`, `or` et `not` combinent les tests [P3].

Après `:`, les lignes du bloc sont indentées : utilise quatre espaces par niveau [P3]. Exemple adapté du tutoriel [P4] :

```
x = int(input("Entier : "))
if x < 0:
    print("negatif")
elif x == 0:
    print("nul")
else:
    print("positif")
```

`input` lit du texte ; `int` le convertit en entier si l'écriture est valide. `print` affiche [P5].

Méthode — *Choisir une boucle*

— `for` parcourt des éléments ; `range` fournit une progression d'entiers. Dans `range(debut, fin)`, la borne `fin` est exclue. Avec une seule borne, le départ est 0 ; un troisième argument fixe le pas [P4].

> — `while` recommence tant que son test est vrai. Le test a lieu *avant* le bloc : celui-ci peut donc ne jamais être exécuté [P3].

Exemple du tutoriel [P4] :

```
for i in range(5):
    print(i)
```

Il affiche les entiers de 0 à 4, chacun sur une ligne. Pour un `while`, prévoir ce qui fera devenir le test faux : une boucle peut sinon tourner sans fin.

9.4 Listes et fonctions

Une liste regroupe des valeurs dans un ordre. Ses indices commencent à 0 : dans `squares = [1, 4, 9, 16, 25]`, `squares[0]` vaut 1 et `squares[-1]` vaut 25 [P1]. `len` donne la longueur ; `append` ajoute un élément à la fin [P5, P6]. Distingue toujours *indice* et *valeur*.

Une fonction reçoit des *paramètres*. `def` introduit sa définition ; `return` renvoie un résultat et arrête l'appel courant [P2, P3]. Afficher avec `print` ne remplace pas `return`.

Exemple 9.2. La suite de Fibonacci commence par 0, 1 ; chaque terme suivant est la somme des deux précédents. Le tutoriel liste les termes strictement inférieurs à une borne [P4]. Adaptation pédagogique :

```
def fibonacci_avant(borne):
    termes = []
    a, b = 0, 1
    while a < borne:
        termes.append(a)
        a, b = b, a + b
    return termes
```

Démonstration. Au début de chaque tour, a et b sont deux nombres de Fibonacci consécutifs. L'affectation simultanée calcule d'abord les deux expressions de droite avec les anciennes valeurs, puis remplace a et b

[P2]. Le passage au couple suivant conserve donc cette propriété. Les valeurs ajoutées respectent $a <$ `borne` ; pour une borne entière fixée, elles finissent par la dépasser. □

> **Méthode** — *Tester sans confondre test et preuve*
>
> Tester un cas ordinaire, un cas limite et une entrée minimale. Vérifier autrement le résultat. Les tests détectent des erreurs ; la preuve traite toutes les entrées admises [A1].

Dans COMIMa, le calcul et la taille des données sont bornés : utilise de petites entrées. Ces exemples ne nécessitent aucune importation.

9.5 Euclide : comprendre un algorithme

> **Définition 9.3.** Un *algorithme* décrit des instructions pour transformer des entrées en un résultat. On précise les hypothèses, puis on vérifie la terminaison et la correction [A1].

Pour $a \geq 0$ et $b > 0$, la division euclidienne donne $a = bq + r$. Un entier divise a et b si et seulement s'il divise b et r. Ainsi $\mathrm{PGCD}(a, b) = \mathrm{PGCD}(b, r)$ [A2].

```
def pgcd(a, b):
    a, b = abs(a), abs(b)
    while b != 0:
        a, b = b, a % b
    return a
```

Entrées : deux entiers non tous deux nuls. **Sortie :** leur PGCD positif. Traduction Python de l'algorithme d'Euclide [A2] ; `abs` permet de traiter aussi les entiers négatifs [P5].

> **Proposition 9.4.** *Cette fonction s'arrête et renvoie le PGCD demandé.*

Démonstration. À chaque tour, la seconde valeur devient le reste, entier compris entre 0 et l'ancienne seconde valeur exclue. Elle décroît donc strictement tant qu'elle est positive : elle finit par atteindre 0. Le PGCD du couple reste inchangé ; à l'arrêt, il vaut le premier nombre, puisque $\mathrm{PGCD}(a, 0) = a$ pour $a > 0$. □

> Un *invariant* est une propriété conservée par les tours de boucle. Un *variant* est ici un entier non négatif qui décroît strictement : il garantit l'arrêt. Ces mots désignent deux questions différentes : « pourquoi le résultat est-il juste ? » et « pourquoi finit-on ? »

Voir aussi les chapitres Récurrence, Invariants et Théorie des nombres.

9.6 Dichotomie : réduire la recherche

Hypothèse essentielle : la liste est triée dans l'ordre croissant. Pour trouver où placer une valeur x, on peut couper la zone de recherche en deux à chaque tour [A3]. Écriture pédagogique de la recherche du premier indice dont la valeur est supérieure ou égale à x :

```
def position(t, x):
    gauche, droite = 0, len(t)
    while gauche < droite:
        milieu = (gauche + droite) // 2
        if t[milieu] < x:
            gauche = milieu + 1
        else:
            droite = milieu
    return gauche
```

La sortie est une *position d'insertion*, éventuellement `len(t)`. Pour savoir si x est présent, vérifier que cette position existe dans la liste et contient x [A3].

Démonstration. À chaque tour, tous les indices strictement à gauche de `gauche` contiennent une valeur $< x$; tous ceux à partir de `droite`

contiennent une valeur $\geq x$. Le tri justifie la moitié éliminée. La différence `droite - gauche` décroît ; à l'arrêt, la position sépare les deux zones. □

Combien d'étapes ?

Un parcours examine jusqu'à n éléments. Après k partages dichotomiques, la zone contient au plus $\lfloor n/2^k \rfloor$ éléments : dix partages suffisent pour 1 000 éléments. On compte les étapes plutôt que des secondes [A1, A3].

Pour une équation, on conserve plutôt un intervalle où une fonction continue change de signe ; l'exercice final précise ce cadre [E10].

9.7 Prolongement : le crible

Pour obtenir tous les nombres premiers jusqu'à n, le crible d'Ératosthène barre des multiples au lieu de recommencer un test complet pour chaque entier [A4]. Pour un entier $n \geq 0$:

```
def premiers(n):
    if n < 2:
        return []
    garde = [True] * (n + 1)
    garde[0] = False
    garde[1] = False
    p = 2
    while p * p <= n:
        if garde[p]:
            for m in range(p * p, n + 1, p):
                garde[m] = False
        p += 1
    return [k for k in range(n + 1)
            if garde[k]]
```

La dernière ligne construit la liste des indices dont l'indicateur est vrai. Adaptation Python de l'algorithme [A4]. Les indices conservés sont les

nombres premiers : 0 et 1 sont exclus. On commence à p^2, car un plus petit multiple composé a déjà un facteur premier plus petit que p. Tout nombre composé $\leq n$ possède un facteur premier $\leq \sqrt{n}$; cela justifie la borne de la boucle. La liste des indicateurs occupe $n+1$ cases.

9.8 À retenir

- `=` affecte ; `==` compare ; l'indentation délimite les blocs.
- `for` parcourt ; `while` répète sous condition.
- Une fonction renvoie un résultat avec `return` ; une liste commence à l'indice 0.
- Pour une boucle : donner un invariant et une raison de s'arrêter.
- Rechercher par dichotomie exige le tri ; les tests complètent la preuve.

9.9 Exercices

Les exercices proviennent des sources citées en fin de chapitre. Les consignes sont adaptées au format papier ; les hypothèses et contraintes utiles sont conservées. Les fonctions demandées sont à écrire, pas à chercher dans un corrigé.

Exercice 9.1 — Fonction affine

Programme une fonction Python qui représente l'application $x \mapsto 4x+7$. Elle reçoit x et renvoie son image.

Exercice 9.2 — Quotient, reste et puissance

Sans exécuter Python, évalue `17 // 3`, `17 % 3` et `2 ** 7`. Vérifie ensuite dans l'interpréteur et distingue les trois opérations.

Exercice 9.3 — Lire range

Écris les listes produites par les expressions suivantes.

1. `list(range(5, 10))`
2. `list(range(0, 10, 3))`
3. `list(range(-10, -100, -30))`

Explique pourquoi la borne finale est exclue.

Exercice 9.4 — Compter un caractère

Construis `compte_occurrences(cible, mot)` : `cible` est un caractère et `mot` une chaîne. Le résultat est le nombre d'apparitions de ce caractère. Parcours la chaîne ; n'utilise pas `count`.

Exercice 9.5 — Additionner les valeurs

Écris `somme_valeurs(tableau)` pour une liste d'entiers. Renvoie la somme de ses éléments, ou 0 si elle est vide. Utilise un accumulateur et une boucle, sans la fonction `sum`.

Exercice 9.6 — Chercher le maximum

Une liste `nombres` est non vide et contient des entiers ou des flottants. Écris `maximum(nombres)` pour en extraire la plus grande valeur. Les fonctions `max`, `sort` et `sorted` sont interdites.

Exercice 9.7 — Premier indice du minimum

Pour une liste non vide, programme `indice_min(tableau)`. Renvoie la position du plus petit élément ; en cas d'égalité, conserve la première position. N'utilise ni `min` ni `index`.

Exercice 9.8 — Lister les diviseurs

Le paramètre n est un entier strictement positif. Programme `diviseurs(n)` pour renvoyer tous ses diviseurs positifs dans l'ordre croissant. Tiens compte de l'absence de diviseur strictement entre $n/2$ et n.

Exercice 9.9 — Tester la primalité

Écris `est_premier(n)` pour un entier $n \geq 0$: renvoie `True` exactement lorsque n est premier. Pour $n \geq 2$, il suffit d'examiner les diviseurs de 2 à $\sqrt{n}$ inclus. Choisis une boucle `for` ou `while`.

Exercice 9.10 — Encadrer une solution

On considère $f(x) = -x^3 + x^2 + x + 1$. Écris `dichotomie(debut, fin, precision)` pour renvoyer un encadrement d'une racine, de largeur au plus `precision`. On suppose `debut < fin`, `precision > 0`, f continue et strictement monotone sur l'intervalle, et $f(\texttt{debut})f(\texttt{fin}) < 0$. Si un milieu est une racine exacte, renvoie ce milieu deux fois.

La précision demandée doit rester atteignable avec les nombres flottants utilisés.

Sources et crédits

Cours : [P1] Python, tutoriel § 3 ; [P2] affectation et return ; [P3] conditions, boucles et fonctions ; [P4] tutoriel § 4 ; [P5] fonctions natives ; [P6] listes. [A1] MIT, Demaine–Ku–Solomon, 6.006, leçon 1 (2020) ; [A2] CP-Algorithms, Euclide ; [A3] Python, bisect ; [A4] CP-Algorithms, crible.

Exercices : [E1] CodEx, Fonctions simples, fonction 0 ; [E2] [P1], § 3.1.1 ; [E3] [P4], § 4.3 ; [E4] Occurrences d'un caractère ; [E5] Somme d'une liste ; [E6] Maximum ; [E7] Indice du minimum ; [E8] Diviseurs ; [E9] Nombre premier ; [E10] Résolution approchée par dichotomie.

Sources consultées le 28 octobre 2026. Exercices CodEx : collectif d'enseignants contributeurs (AEIF), CC BY-SA 4.0. Adaptations COMIMa sous la même licence ; extraits de code Python sous licence 0BSD (documentation Python). La progression s'appuie sur les ressources lycée Éduscol. Références détaillées et nature des adaptations : `docs/sources/PY1.md`.